Pokémon RL Midsem Plan

Introduction

We want to measure how an RL algorithm learns different *playing strategies* in doubles VGC. Specifically, we compare a standard team with a high-risk, high-reward team such as Perish Trap.

That difference is hard to quantify directly. Perish Trap differs from a standard team in many ways at once: which moves matter, multi-turn timing, trapping and when to switch. Any result would be hard to attribute to a single cause.

So the midsem starts with a simpler difference that is easy to show and fully controlled: **variance**. We take one 6v6 singles team and build two versions that are identical except for a few moves. In the high-variance version, those moves are swapped for lower-accuracy, higher-base-power versions (e.g. Surf → Hydro Pump). Pokémon, items, abilities, EVs, opponents and algorithm settings all stay the same.

We then study how this one change affects *how the agent learns*, using three measures:

- **Sample efficiency:** how much experience it needs to get good.
- **Stability:** how consistent learning is across random seeds and over training.
- **RL lift:** how much the trained agent improves over a scripted player using the same team.

We do this for two kinds of RL, on both team versions:

- **DQN (deep Q-learning):** value-based; it learns a score for each move.
- **PPO:** policy-based; it learns move probabilities directly.

For the midsem, the only opponent is poke-env's built-in heuristic player. The final moves to 4v4 doubles VGC with the standard vs Perish Trap comparison. It reuses this pipeline and these metrics, and adds existing bots as opponents (Section 9).

1. Goal and research questions

The midsem trains DQN and PPO agents on one 6v6 Gen 9 OU singles team in two versions that differ only in move accuracy and power, and compares how fast and how reliably each algorithm learns each version. Presentation: Wed 21 Oct 2026.

- **RQ1 (variance → learning):** Does the high-variance (inaccurate-move) team need more training battles to reach the same performance, and do its runs disagree more across seeds?
- **RQ2 (algorithm × variance):** Does DQN degrade more than PPO on the high-variance team? Theory predicts yes (Section 5).
- **RQ3 (strength):** Does the final win rate differ? This is secondary, because the swap also changes damage thresholds (Section 2).
- **Deliverable 2 (spec):** MDP, value function and policy definitions that carry over to the 4v4 VGC final.

Honest framing for the slides: this is a pilot. Expect 2–3 seeds per condition and wide error bars. The deliverable is a working pipeline, a sound protocol and first evidence, not a definitive result.

2. Experimental design and verification of the move swap

Verdict: the swap works and is a much cleaner experiment than two different teams. It is not a pure variance change, though, so the slides must report the expected-damage shift and control for it.

Setup

- Gen 9 singles, 6v6, preset teams, a fixed lead (slot 1) and no team-preview decision.
- Tera disabled for **both** players, so the action space is moves plus switches only.
- Our team comes in two versions, LowVar and HighVar, identical in species, items, abilities, EVs, natures and every non-swapped move.
- 3 preset opponent teams, piloted only by poke-env's `SimpleHeuristicsPlayer`. Existing strong bots are for the final (Section 9).
- Each training episode draws an opponent team uniformly at random. This is identical for all conditions.

Why the swap is good design. Holding everything else fixed means any difference in learning comes from the changed moves. In stats terms the moves are the single manipulated variable. The 4 untouched Pokémon act as within-team controls.

Candidate swap pairs and their expected damage (base power × accuracy). You pick the pairs you'll actually use in Section 3. If you use a pair not listed here, compute its expected BP the same way.

Low-variance move	High-variance move	Expected BP	Change	Other differences
Surf 90 / 100%	Hydro Pump 110 / 80%	90 → 88	−2%	PP 24 → 8
Flamethrower 90 / 100%	Fire Blast 110 / 85%	90 → 93.5	+4%	PP 24 → 8; burn chance 10% in both
Thunderbolt 90 / 100%	Thunder 110 / 70%	90 → 77	−14%	Paralysis 10% → 30%; PP 24 → 8
Ice Beam 90 / 100%	Blizzard 110 / 70%	90 → 77	−14%	Freeze 10% in both; PP 24 → 8

PP values are maxed (PP Ups); verify in the teambuilder.

Caveats to state on the slides

1. **Not mean-preserving.** Thunder and Blizzard also lower expected damage by 14%. Hydro Pump and Fire Blast are close to a pure variance change.
2. **KO thresholds.** 110 BP can turn a 2HKO into an OHKO, so in some matchups the inaccurate move is *better* in expectation. Raw win rate therefore mixes strength and variance. The fix is the RL-lift metric in Section 6: compare each RL agent to a heuristic piloting the same team version.
3. **Weather breaks the manipulation.** Snow makes Blizzard 100% accurate. Rain makes Thunder 100% accurate, and sun drops it to 50%. Opponent teams must contain no weather setters. Slowking-Galar's Chilly Reception sets snow and it is a top-20 OU Pokémon, so exclude it.

4. **Accuracy modifiers** also break it: Compound Eyes, No Guard, Wide Lens, Bright Powder, accuracy or evasion boosts. None of these may appear on any team.
5. **PP.** 8 PP rarely runs out, but 6v6 battles can go 40+ turns, so log it.
6. **Secondary effects.** Thunder's higher paralysis chance is a second source of randomness. It's acceptable because it is also variance, but name it.

Optional cleaner sub-test (final project). A "strict" HighVar team that swaps only Surf→Hydro Pump and Flamethrower→Fire Blast (nearly mean-preserving). The effect will be small, though, and probably not detectable with midsem seed counts.

A subtle point about "variance". A win/loss outcome has variance $p(1-p)$, which depends only on the win rate. So "high variance" here does not mean the final result is more random. It means each *transition* is noisier: the same state and action lead to more spread-out next states. That is exactly what makes TD targets and policy-gradient estimates noisier, which is the mechanism we test.

3. Teams (you fill these in)

You choose the teams and exact sets. Do it in this order:

1. Pick the swap pairs.
2. Build our team around Pokémon that legally learn *both* moves of a pair.
3. Build the 3 opponent teams.

Check every set with the Showdown teambuilder validator for the format you choose in task T5; illegal moves are flagged there. Current usage data to draw from: PokeTools Gen 9 OU, September 2026.

Swap pairs (pick 3–4)

Expected BP = base power × accuracy. Pairs with similar expected BP isolate variance best (Section 2). Candidates are in the Section 2 table.

#	LowVar move (BP / acc)	HighVar move (BP / acc)	Expected BP, low → high	Used by
1				
2				
3				
4				

Our team

Both versions are identical except the swapped move. For control Pokémon (no swap), list all 4 moves under Shared moves and leave the last two columns as —. Slot 1 is the lead in every battle.

Slot	Pokémon	Item	Ability	Nature / EVs	Shared moves	LowVar move	HighVar move
1 (lead)							

Slot	Pokémon	Item	Ability	Nature / EVs	Shared moves	LowVar move	HighVar move
2							
3							
4							
5							
6							

Rules for our team

- At least 3 of the 6 Pokémon carry a swap. The rest act as controls.
- No weather setters (Drizzle, Drought, Snow Warning, Chilly Reception, Rain Dance, Sunny Day, Snowscape): they change Thunder, Blizzard and Hurricane accuracy.
- No accuracy or evasion modifiers: Compound Eyes, No Guard, Hustle, Victory Star, Wide Lens, Zoom Lens, Bright Powder, evasion- or accuracy-boosting moves.
- Avoid swap moves with big self-drawbacks (Draco Meteor, Leaf Storm, High Jump Kick crash damage). They change strategy, not just variance.

Opponent teams (3)

Aim for 3 different styles (e.g. balance, offense, bulky). Same weather and accuracy rules as above, and no Pokémon that is also on our team.

Opponent A — style:

Pokémon	Item	Ability	Moves	EV Spread

Opponent B — style:

Pokémon	Item	Ability	Moves	EV Spread

Opponent C — style:

Pokémon	Item	Ability	Moves	EV Spread

Opponent policy (midsem)

Training uses only poke-env's `SimpleHeuristicsPlayer` (check the exact name in your installed version). Evaluation also includes random and max-base-power players as simple sanity baselines. These are not "existing bots"; those come in the final.

Diff check

A 10-line script parses both team files and asserts that only the chosen swap moves differ. Put its output on a slide: it is the proof that everything else was held constant.

If you want a default to start from: Kingambit and Great Tusk as controls; Primarina (Surf ↔ Hydro Pump), Kyurem (Ice Beam ↔ Blizzard), Raging Bolt (Thunderbolt ↔ Thunder) and Moltres (Flamethrower ↔ Fire Blast) as treated. These are based on September 2026 OU usage, but the learnsets are from memory and unverified.

4. MDP specification: value function and policy

This is the formal part of the midsem deliverable. The same definitions extend to doubles in the final; only the action space and some features change.

State s (observation vector, roughly 150–250 numbers, all scaled to about [0, 1])

- For each of our 6 Pokémon: HP fraction, fainted flag, status one-hot (none / brn / par / psn / tox / slp / frz) and an active flag.
- For each of the opponent's 6 Pokémon: the same, using only what has been revealed. This is partial observability.
- For our active Pokémon: 7 stat-boost stages (/6), 2 types (one-hot of 18).
- For each of its 4 moves: base power/150, accuracy, type effectiveness vs the opponent's active, STAB flag, PP fraction.
- For the opponent's active Pokémon: boosts and types.
- Field: hazards on each side (Stealth Rock, Spikes layers, Toxic Spikes, Sticky Web), screens, and turn/100.

Including **accuracy** as a feature matters for this experiment. Without it the agent cannot tell Surf from Hydro Pump except through their outcomes.

Action a. The discrete action space of poke-env's `SinglesEnv`: 4 moves plus 5 switches are the legal subset, and gimmick actions (Tera, Mega, Z, Dynamax) are always masked. An action mask removes illegal actions at every step.

Reward r. A terminal reward of +1 for a win and −1 for a loss, plus potential-based shaping:

$$r'_t = r_t + \gamma \Phi(s_{t+1}) - \Phi(s_t), \quad \Phi(s) = 0.5\,\frac{\text{HP}_{ours} - \text{HP}_{opp}}{6} + 0.5\,\frac{\text{alive}_{ours} - \text{alive}_{opp}}{6}$$

Shaping of this form provably leaves the optimal policy unchanged (Ng, Harada and Russell, 1999, from memory). It just gives feedback before the battle ends. Always multiply the next potential by γ; forgetting that is a common bug.

Discount. γ = 0.99. The effective horizon is about 1/(1−γ) = 100 decisions, which covers a 30–60 turn 6v6 battle.

Value functions (what both algorithms estimate)

$$V^{\pi}(s) = \mathbb{E}_{\pi}\Big[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \,\Big|\, s_t = s\Big] \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}\Big[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \,\Big|\, s_t = s, a_t = a\Big]$$

In words, V(s) means "how good is this board for me if I keep playing my way". Q(s, a) means "how good is it if I play a now, then my way".

Policy 1: DQN (value-based). Learn Q with a neural network using the Bellman optimality target. The policy is derived from Q: ε-greedy over legal actions, with ε decaying from 1.0 to 0.05 over the first 30% of training, and greedy at evaluation.

$$y_t = r_t + \gamma \max_{a' \in \mathcal{A}_{legal}(s_{t+1})} Q_{\bar\theta}(s_{t+1}, a') \qquad \mathcal{L}(\theta) = \big(y_t - Q_\theta(s_t, a_t)\big)^2$$

Here θ^- is a slowly updated target network, and transitions are sampled from a replay buffer. The max must also respect the mask: illegal actions get $-\infty$.

Policy 2: PPO (policy-based, actor-critic). An actor outputs $\pi(a|s)$ as a masked softmax. A critic learns $V(s)$. Updates push up the probability of actions with positive advantage A, with clipping so one update cannot change the policy too much.

```
L^{CLIP}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t A_t,\ \text{clip}(\rho_t,
1-\epsilon, 1+\epsilon) A_t\big)\Big], \quad \rho_t =
\frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}
```

A_t is computed with GAE ($\lambda = 0.95$), and $\varepsilon = 0.2$.

Fairness rules between the algorithms

- Same encoder, same reward, same network body (e.g. 2 hidden layers of 256), same number of environment steps, same opponents, same seeds.
- Default-ish hyperparameters for both. Do not tune one more than the other.
- Evaluate both deterministically (DQN greedy, PPO argmax).

Simplifications for the midsem, stated explicitly

- Fixed teams and lead, no Tera, one fixed opponent policy.
- Partial observability is ignored: no memory or recurrence, just the current observation.

5. Algorithms: Q-learning vs PPO and expected results

The main prediction is an *interaction*: DQN should lose more performance than PPO when moving from LowVar to HighVar. There is a clear theoretical reason for this, so it is a good hypothesis to test.

"Q-learning" here means DQN. Tabular Q-learning needs a table entry per state. Pokémon has astronomically many states, so a neural network must approximate Q. That is DQN: the same Q-learning update with a function approximator, a replay buffer and a target network.

Masking problem with DQN. Stable-Baselines3's `MaskablePPO` lives in sb3-contrib (docs). There is no maskable DQN there. The options are:

1. **Recommended:** write a small masked DQN in PyTorch (about 150 lines). Masking goes in both action selection and the max in the target. A one-line change turns it into Double DQN. It is also the best learning exercise in the project.
2. Fallback: SB3 DQN, where an illegal choice is replaced by a random legal move. Simple, but it adds noise and favours PPO, so label it as a confound if used.

Expected results and why

Prediction	Reason	How we'll see it
DQN overestimates action values more on HighVar	Q-learning uses the max over noisy estimates, which is biased upward; van Hasselt showed this makes Q-learning perform very poorly in	Plot mean predicted max-Q at battle start vs actual discounted return; the gap should be larger on HighVar

Prediction	Reason	How we'll see it
	some stochastic environments (Double Q-learning, NeurIPS 2010)	
PPO is more stable across seeds	Clipped, on-policy updates and a learned baseline (the critic) reduce gradient variance	Smaller spread between seed curves
DQN can be more sample-efficient early on LowVar	Replay reuses each transition many times, while PPO discards data after each update	DQN curve rises first on LowVar, if anything
Both learn slower on HighVar	Noisier TD targets and advantages, so more samples per unit of signal	Later threshold crossing, lower learning-curve area
PPO critic explains less variance on HighVar	Returns are less predictable from the state	SB3's logged `explained_variance` is lower
Final win rates may barely differ	Stronger 110-BP hits partly offset misses (Section 2)	Use RL lift, not raw win rate

Honest expectations for the midsem

- With about 100k–300k steps and 3 seeds per cell, differences may sit inside the noise. Report that plainly.
- The DQN pipeline is the biggest schedule risk. If it isn't stable by Monday morning, present PPO results plus DQN as preliminary.
- These are theory-based predictions, not results anyone has published for Pokémon specifically. I found no Pokémon study comparing these algorithms across accuracy variants.

Optional third arm (only if time): Double DQN. If the overestimation story is right, Double DQN should close most of DQN's gap on HighVar. That would be a strong, clean finding for the final.

6. Metrics: what sample efficiency and stability measure

Both metrics measure **how hard a team is to learn for a given algorithm**, not how strong the team is. Keep those two claims separate in the slides.

Metric	Exact definition	What it tells us	What it does not tell us
Sample efficiency: threshold	Environment steps until the evaluation win rate vs the heuristic first stays above a target (e.g. heuristic baseline + 10 points) for 2 consecutive evals	How much experience the algorithm needs to extract a useful signal from this team	Final strength; it is sensitive to the chosen threshold

Metric	Exact definition	What it tells us	What it does not tell us
Sample efficiency: AUC	Area under the eval win-rate curve divided by total steps (the average win rate over training)	Same idea, with no threshold to choose; more robust with few seeds	Where learning plateaus
Stability: across seeds	Standard deviation (or range) of final win rate over seeds	How much the outcome depends on luck in initialisation and exploration, i.e. how reproducible the method is on this team	Anything reliable with only 3 seeds; show every seed curve, not just a mean
Stability: within run	Largest drop between consecutive evals, and std of the last 5 evals	Whether learning oscillates or collapses (common for DQN)	The cause of the oscillation
RL lift	Final RL win rate minus win rate of the heuristic piloting the same team version	How much learning adds beyond scripted play, which cancels most of the strength difference from the swap	Absolute team strength
Diagnostics	DQN: predicted max-Q vs realised return. PPO: critic explained variance. Both: miss rate, battle length	The *mechanism* behind any difference	—

Measure the x-axis in environment steps (decisions), and also report battles. HighVar battles may be longer because misses prolong them, so steps and battles diverge.

Statistics

- Win rate over N battles: report a Wilson 95% confidence interval. N = 500 gives about ±4.4 points near 50%, and N = 1000 gives about ±3.1.
- With 3 seeds, show every individual curve. The deep-RL evaluation literature warns that point estimates from a handful of runs are unreliable and recommends interval estimates and robust aggregates such as the interquartile mean (Agarwal et al., NeurIPS 2021). IQM needs more runs than you'll have, so cite it as the method for the final.
- Headline figure: a 2×2 interaction plot (algorithm × team), with AUC on the y-axis and one dot per seed.

7. Step-by-step task list (from Sunday morning)

It is Sunday 18 October, morning. That leaves today, Monday and Tuesday before the Wednesday presentation.

The **critical path** is T2 → T3 → T6 → T10/T11 → T13 → T14 → T15 → T20 → T21. Anything on it that slips moves the deadline, so the pipeline work (T6, T10, T11) must finish today so the main runs can train overnight. The biggest single risk is T11 (DQN).

Tasks without a dependency on each other can run in parallel, so split them between the two of you. A natural split is one person on the training pipeline and one on teams, baselines, evaluation and slides. Fill in Owner together.

ID	When	Task	Needs	Est.	Owner
T1	Sun AM	Choose teams and fill Section 3	—	1–1.5 h	
T2	Sun AM	Set up the environment (both laptops)	—	1–2 h	
T3	Sun AM	Pin versions, read the API, run the official RL example	T2	1 h	
T4	Sun midday	Team files, validation, diff script	T1	1 h	
T5	Sun midday	Battle format, Tera off, team rotation, first real battle	T2, T4	1 h	
T6	Sun PM	Observation encoder	T3	3 h	
T7	Sun PM	Reward function	T3	1 h	
T8	Sun PM	Baselines (runs in background)	T5	1 h + runtime	
T9	Sun PM	Throughput benchmark and step budget	T6	1 h	
T10	Sun evening	PPO pipeline	T6, T7	3 h	
T11	Sun evening	Masked DQN pipeline	T6, T7	4–5 h	
T12	Sun evening	Checkpoint evaluation script	T5	2 h	
T13	Sun night	Smoke runs: all 4 cells, short	T9–T12	1 h + runtime	
T14	Sun night → Mon	Main training runs (12)	T13	runtime	
T15	Mon	Checkpoint evaluations for learning curves	T12, T14	runtime	
T16	Mon	Final-project spec	T6	3 h	
T17	Mon	Theory and related-work slides	—	3 h	
T18	Mon	Doubles smoke test	T2	1 h	

ID	When	Task	Needs	Est.	Owner
T19	Tue AM	Final evaluations	T14	runtime	
T20	Tue	Analysis and plots	T8, T15, T19	3 h	
T21	Tue	Slides, replays and rehearsal	T16, T17, T20	3 h	

What each task involves

T1 — Choose teams and fill Section 3. Pick the swap pairs first, then the Pokémon that legally learn both moves, then the opponents. Type each set into the Showdown teambuilder as you go so illegal moves show up immediately. This comes first because the team files, baselines and training all depend on it, and it's the one decision you need to make together.

- *Done when:* every table in Section 3 is filled and both of you agree.

T2 — Set up the environment. Do this on both laptops, so training can be split across two machines later.

```
sudo pacman -S nodejs npm git
git clone https://github.com/smogon/pokemon-showdown.git
cd pokemon-showdown && npm install && cp config/config-example.js config/config.js
node pokemon-showdown start --no-security   # leave running in its own tmux pane
```

Then the Python side. poke-env needs Python 3.10 or newer; 3.11 avoids PyTorch wheel gaps on Arch's newest Python.

```
uv venv --python 3.11 .venv && source .venv/bin/activate
uv pip install poke-env stable-baselines3 sb3-contrib torch tensorboard
uv pip freeze > requirements.txt   # share this file so both laptops are identical
```

- *Done when:* the "first local battle" script from the poke-env README (two RandomPlayers) finishes on both machines.

T3 — Pin versions, read the API, run the official example. poke-env changes between versions, so read your installed source rather than blog posts. Write down five things in a shared notes file:

1. The exact action-space size.
2. Which action index means which move or switch (`action_to_order`).
3. How the action mask is exposed.
4. The heuristic player's class name.
5. How a `Teambuilder` supplies teams.

Then run poke-env's official reinforcement-learning example unchanged. It already trains PPO against `SimpleHeuristicsPlayer` with action masking, so it is the template for T10: your pipeline is that example plus your encoder, reward and teams.

- *Done when:* the example runs end to end and the five notes are written.

T4 — Team files, validation, diff script. Export each team from the teambuilder into `teams/lowvar.txt`, `teams/highvar.txt` and `teams/opp_a.txt`, `opp_b.txt`, `opp_c.txt`. Validate all five in the format chosen in T5. Write the diff script (Section 3).

- *Done when:* all five validate and the diff script reports only the intended swaps.

T5 — Battle format, Tera off, team rotation, first real battle.

- **Tera must be off for both players.** Masking only our Tera actions would leave the heuristic opponent free to Terastallize. Find a format or rule setup where Tera is banned for everyone.
- If that's not possible through poke-env, mask our Tera actions *and* subclass the opponent so it never Terastallizes. Write down which route you took.
- Write a small `Teambuilder` subclass that returns one of the 3 opponent teams at random each battle.
- Leave team order as given, so slot 1 always leads.
- *Done when:* 20 test battles run with slot 1 always leading, no `-terastallize` lines in the logs, and all 3 opponent teams appearing.

T6 — Observation encoder. Subclass `SinglesEnv` and implement `embed_battle` following Section 4. Declare the observation space with correct bounds and scale every feature to roughly [0, 1] (or [−1, 1] for boosts).

- Include **move accuracy**. Never-miss moves are reported as `True`, so convert them to 1.0. Without accuracy, the agent can't tell the two team versions apart except through outcomes.
- *Done when:* a test over 20 random battles shows a fixed vector length, no NaN or infinite values, and everything within bounds.

T7 — Reward function. Implement `calc_reward` as a terminal ±1 plus the potential-based shaping from Section 4. Store the previous potential per battle, and remember the γ in front of the next potential.

- *Done when:* hand-checking 2–3 battles shows the final step carries ±1 with the right sign and the shaping terms are small (well under 1 per step).

T8 — Baselines. Pilot each of our team versions with random, max-base-power and heuristic players against the heuristic opponent, 500 battles each (6 cells, 3,000 battles; let it run in the background). Record:

- Win rate with Wilson confidence interval.
- Mean battle length.
- Miss rate (count `-miss` lines in the logs).

This is your first real result: it shows how much the swap changed strength and noise before any learning. The heuristic-with-our-team win rate is also the reference for RL lift.

- *Done when:* a CSV with all 6 cells exists.

T9 — Throughput benchmark and step budget. Run a random agent through your env for 2 minutes each with 1, 4 and 8 parallel envs, and measure steps per second. Then set the step budget so 12 runs fit overnight on two laptops, using the formula in Section 8, and write the number there. The simulator is CPU-bound, so the GPU barely matters.

- *Done when:* the step budget is written in Section 8.

T10 — PPO pipeline. Start from the T3 example and swap in your env, encoder, reward and team rotation.

- Settings: learning rate 3e-4, about 2,048 steps per update, batch 128, γ = 0.99, GAE λ = 0.95, clip 0.2, entropy coefficient 0.01, network 256-256.
- Save a checkpoint every 10k steps and log each training episode's win or loss.
- *Done when:* a 20k-step sanity run against `RandomPlayer` shows a clearly rising win rate.

T11 — Masked DQN pipeline. Write it in PyTorch, with the same 256-256 network body as PPO for fairness. You need:

- A replay buffer (about 100k transitions) and a target network (synced every 2,000 steps).
- Learning rate 1e-4, batch 64, γ = 0.99, learning starts after 5k steps.
- ε decaying from 1.0 to 0.05 over the first 30% of steps.
- The mask applied in **two** places: set illegal Q values to $-\infty$ before choosing an action, and before taking the max in the target.
- No bootstrapping on the final step of a battle.
- An optional `double=True` flag that picks the next action with the online network but values it with the target network.
- Log the predicted max-Q at each battle's first state alongside the actual discounted return; that is the overestimation plot.
- *Done when:* the same 20k-step sanity run shows a rising win rate.

T12 — Checkpoint evaluation script. Load any checkpoint (PPO or DQN), act **deterministically** (DQN greedy, PPO argmax), play N battles against a chosen opponent with the same team rotation, and append one CSV row: algorithm, team, seed, step, opponent, wins, N, Wilson low/high, mean length.

- Deterministic play matters because training win rates aren't comparable between algorithms: DQN trains with ε-random moves, PPO with sampled moves.
- *Done when:* it works on a T10 and a T11 checkpoint.

T13 — Smoke runs. Run all 4 cells (2 algorithms × 2 teams), 1 seed each, about 20k steps, then evaluate their checkpoints with T12. This catches crashes, NaN losses and wiring bugs before you spend the night training.

- *Done when:* all 4 finish and evaluation produces sensible numbers.

T14 — Main training runs. 12 runs: 2 algorithms × 2 teams × seeds 0, 1, 2, all at the same step budget. Split the runs across both laptops only if `requirements.txt` and the code are identical on both. Use one tmux window per batch and log to `runs/<algo>_<team>_s<seed>/`.

- Rule: **never change code or settings mid-batch.** If a bug fix is needed, restart every affected run so all cells stay comparable.
- Note: seeds control network initialisation and exploration, not Showdown's battle RNG.
- *Done when:* all 12 runs have finished and saved checkpoints.

T15 — Checkpoint evaluations for learning curves. Evaluate every 20k-step checkpoint of every run, 100 battles each against the heuristic. These points are the learning curves for sample efficiency and within-run stability. Start on finished runs while others are still training.

- *Done when:* every checkpoint has a CSV row.

T16 — Final-project spec. Doubles 4v4 MDP:

- `DoublesEnv` per-slot actions with targets.
- Extra features (perish counter, trapped flag, partner state).
- The reward.
- Opponents: heuristic first, then a mix with existing bots (Section 9).
- The same metrics.

This is half the midsem deliverable.

T17 — Theory and related-work slides. MDP and Bellman equation, how DQN and PPO learn, overestimation bias and why it predicts the DQN × HighVar interaction, then VGC-Bench, Metamon and the PokéAgent Challenge.

T18 — Doubles smoke test. Two random players through `DoublesEnv` for a few battles. One slide's worth of proof that the final is technically feasible.

T19 — Final evaluations. The last checkpoint of each of the 12 runs: 1,000 battles against the heuristic (drop to 500 if short on time), plus 200 each against random and max-base-power.

T20 — Analysis and plots.

- Learning curves with every seed shown.
- Threshold steps and AUC per cell.
- Seed spread and largest within-run drop.
- RL lift: final win rate minus the T8 heuristic baseline for the same team.
- The 2×2 interaction plot.
- DQN's predicted-Q vs actual-return plot, and PPO's explained variance.
- One summary table per cell.

T21 — Slides, replays and rehearsal. Suggested order:

1. Introduction and motivation.
2. RL basics.
3. Setup and swap verification (the expected-BP table and diff check).
4. The MDP spec.
5. Algorithms and predictions.
6. Baselines.
7. Results.
8. Honest limitations.
9. Plan for the final.

Save 2 replays: a game lost to a Hydro Pump-style miss, and a clean LowVar win. Rehearse once with a timer.

8. Compute budget, cut lines and risks

The step budget is set after T9. I don't know the real throughput on your machine, so the numbers below are placeholders to replace.

Budget formula. Hours per batch = steps per run ÷ (steps per second per run × 3600). Example: 200k steps at 50 steps/s comes to about 1.1 h per run. With 4–6 runs in parallel on the Ryzen's 8 cores, 12 runs take about 2–4 h. Several runs can share one Showdown server, but start a second server on another port if it becomes the bottleneck.

Cut lines, in order, if time runs short

1. Drop to 2 seeds per cell. This is the minimum for any stability statement.
2. Halve the step budget, keeping it equal across all cells.
3. Drop the Double DQN arm (it is optional anyway).
4. Present PPO on both teams plus DQN as preliminary. Never drop a team: the team comparison is the point.

Risk	Sign	Fallback
DQN doesn't learn	Win rate vs random not rising after 50k steps	Check masking in the target, reward scale and target-network update rate; else use SB3 DQN with legal-move fallback and label it a confound
Low throughput	Below 20 steps/s	Fewer steps, fewer seeds, run overnight
Neither algorithm beats the heuristic	Flat curves vs the heuristic	Present learning vs random plus the baselines from T6; still a valid pilot
poke-env API mismatch	Code from examples fails	Follow the examples folder of the installed version
A learnset is illegal	Validator error	Replace that Pokémon with another OU user of the same swap pair

9. How this carries into the final, and sources

The final reuses the encoder, reward, both training pipelines, the evaluation script and the metrics. It changes to `DoublesEnv`, 4v4 with fixed teams and per-slot actions with targets. It adds the Perish Trap vs standard comparison as a second axis: strategic complexity rather than RNG variance. VGC-Bench's doubles action encoding is the reference implementation to study (VGC-Bench).

Existing bots as opponents (final only)

- **Curriculum.** Learn the basics against the heuristic player first, then train against a mix (e.g. 80% heuristic, 20% an existing bot), sampling the opponent per battle the same way opponent teams are sampled now.

- **Candidates.** Metamon's heuristic baselines and pretrained agents (Gen 9 OU supported) (Metamon). Foul Play, a tree-search bot built on poke-engine (poke-engine), run at a fixed search time per turn so its strength stays constant.
- **Held-out evaluation.** Evaluate final checkpoints against a strong bot the agent never trained on, to test whether the variance effect holds against good play and whether the agent overfit to the heuristic.
- **Hypothesis to test: variance helps the underdog.** Against an opponent stronger than our agent, the HighVar team may do relatively *better*, because a weaker player needs luck to win. A flip between the heuristic and strong-bot results would refine the original "low variance is better" idea.
- **Practical limits.** Strong bots think per turn, so they are far slower than the heuristic. Keep them for evaluation and a small training share, and always give them our preset opponent teams so only one thing changes.

Sources (opened while writing this plan)

- poke-env documentation: env module
- sb3-contrib overview (MaskablePPO)
- van Hasselt, Double Q-learning (NeurIPS 2010)
- Agarwal et al., Deep RL at the Edge of the Statistical Precipice (NeurIPS 2021)
- Angliss et al., VGC-Bench (AAMAS 2026)
- Grigsby et al., Metamon (RLC 2025)
- PokeTools Gen 9 OU usage, September 2026

Cited from memory, not opened: Ng, Harada and Russell (1999) on potential-based shaping; Mnih et al. (2015) on DQN; Schulman et al. (2017) on PPO. Move base power, accuracy and PP values are also from memory, so verify them in Showdown.

Own Notes:

- Sparse Reward Problem: Each game is about 30-50 turns, decisions (actions to be taken) need to be made each turn, but reward is gotten only at end of battle